

企业级应用软件开发与开发期末大作业报告

课程名称	企业级应用软件开发与开发
项目名称	MiniAgent：面向真实文件任务的可验证 Agent Runtime 与 Harness 系统
方向	方向一：Agentic AI 原生开发
学号	[待填写]
姓名	[待填写]
专业	[待填写：计算机技术 / 软件工程]
指导教师	戚欣
提交日期	2026 年 6 月 22 日

草稿提示：个人信息、课程总结与截图仍需在提交前补充。

目录

一、选题背景与设计思想	3
1.1 问题背景	3
1.2 现有方案不足	3
1.3 项目价值	3
1.4 技术路线	3
二、Specs 规格文档	3
2.1 Product Spec	4
2.2 Architecture Spec	4
2.3 API Spec	4
三、系统架构与设计	4
3.1 系统架构	4
3.2 Agent 交互流程	5
3.3 多 Channel 消息设计	5
3.4 数据流设计	5
四、关键实现与代码展示	5
4.1 Agent 核心循环	5
4.2 Tool 与 Skill Runtime	6
4.3 RuntimeVerifier 与 RuntimeRecovery	6
4.4 AI IDE 使用	6
五、测试与评估	6
5.1 Agent Benchmark	6
5.2 Memory Retrieval	7
5.3 Replay 与失败分析	7
六、系统升级与扩展	7
6.1 MCP-Compatible Tool Adapter	7
6.2 Attachment Index	7

6.3 User-Scoped Memory7

6.4 Workflow Planner 与沙箱7

6.5 云端部署7

七、课程总结 7

参考资料与引用 8

一、选题背景与设计思想

1.1 问题背景

大语言模型具备自然语言理解和工具选择能力，但在真实任务中仍可能出现“语言上声称完成、实际上没有执行”的问题。例如：模型根据文件名猜测内容、没有调用工具却声称已读取文件、脚本失败后仍声称成功、给出并不存在的输出路径。

这些问题表明，Agent 系统不能只依赖提示词和模型自觉，还需要一个工程化 Runtime 对真实证据、执行状态和最终产物进行管理。

1.2 现有方案不足

- 普通 ReAct Agent 通常将模型最终回复直接返回用户，缺少事实校验。
- Tool Calling 只解决“模型能够调用工具”，不保证模型一定调用必要工具。
- Skill 路由命中不等于脚本真实执行。
- 文件任务只检查回复文本，无法判断文件是否真实生成、内容是否正确。
- 开发、评测和线上运行使用不同执行路径，导致评测结果不能代表真实行为。

1.3 项目价值

MiniAgent 构建了统一的 Agent Runtime 与 Harness：

- 统一 CLI、QQBot 和评测入口。
- 支持真实文件输入、Skill 和工具调用。
- 使用 TurnIntent 描述本轮工程约束。
- 使用 RuntimeVerifier 校验文件证据、脚本执行和输出产物。
- 使用 RuntimeRecovery 按错误类型恢复重试。
- 使用 Trace、Benchmark、Replay 和 Regression 建立可观测评测闭环。

1.4 技术路线

多 Channel 消息接入
-> TurnIntent 与上下文构建
-> Skill / Memory / Tools
-> ReAct Agent Loop
-> RuntimeVerifier
-> RuntimeRecovery
-> Trace / Benchmark / Replay / Regression

二、Specs 规格文档

本项目采用 SDD 规格驱动开发方法，将需求、架构、接口和测试建立映射关系。

2.1 Product Spec

完整文档见 docs/specs/product-spec.md。

核心需求：

- 涉及文件内容时必须具有真实读取证据。
- 要求输出文件时必须产生真实 outbox 产物。
- 需要脚本型 Skill 时必须成功执行 run_skill_script。
- 工具失败或中间态回复不得被当作成功结果。
- live 与 eval 必须复用同一套 Runtime。

2.2 Architecture Spec

完整文档见 docs/specs/architecture-spec.md。

架构原则：

- Channel、Runtime、Execution、Control、State 和 Harness 分层。
- Verifier 只负责校验，Recovery 只负责恢复。
- Runtime 控制面不依赖模型自然语言承诺。

2.3 API Spec

完整文档见 docs/specs/api-spec.md。

核心协议：

- InboundMessage / OutboundMessage。
- Tool / ToolRegistry。
- TurnIntent。
- VerificationResult / RecoveryPlan。
- Runtime trace JSONL event。
- Benchmark assertion。

三、系统架构与设计

3.1 系统架构

完整架构图见 ARCHITECTURE.md。

[待添加：系统总览图截图]

3.2 Agent 交互流程

MiniAgent 在标准 ReAct 工具循环外增加了 Intent、Verifier 和 Recovery：

User Request
-> TurnIntent
-> Memory / Skill / Attachment Context
-> Deterministic Action or ReAct Loop
-> Tool Result
-> RuntimeVerifier
-> RuntimeRecovery
-> Final Reply

[待添加：Runtime Turn Flow 时序图截图]

3.3 多 Channel 消息设计

CLI 和 QQBot 都通过 BaseChannel 转换为统一 InboundMessage，再发布到 MessageBus。Agent 完成任务后发布 OutboundMessage，由原 Channel 负责发送。

这一设计使 Agent Runtime 不依赖具体通信平台，新增 Channel 时无需修改核心 Agent Loop。

3.4 数据流设计

- 上传文件写入 inbox。
- 生成文件写入 outbox。
- 短期消息写入 session JSONL。
- 长期事实写入 memory store。
- 运行事件写入 runtime trace。
- Skill 脚本执行写入 skill trace。

四、关键实现与代码展示

4.1 Agent 核心循环

src/miniagent_core/app.py 中的 agent_loop() 实现：

- LLM 请求与 tool schema 注入。
- Tool call 参数解析和执行。
- Tool result 回注。
- 最大迭代控制。
- 最终回复校验和恢复重试。

[待添加：agent_loop 关键代码截图]

4.2 Tool 与 Skill Runtime

ToolRegistry 为本地工具提供统一注册和执行协议。项目级 Skill 通过 SKILL.md 描述能力，通过 run_skill_script 执行受控脚本。

确定性单步任务可由 actions.json 直接生成执行计划，降低模型自由规划的不稳定性。

[待添加：ToolRegistry、run_skill_script、actions.json 截图]

4.3 RuntimeVerifier 与 RuntimeRecovery

Verifier 校验：

- 文件读取证据。
- 输出文件产物。
- Skill 脚本成功。
- 虚假完成声明。
- 不完整进度回复。

Recovery 根据 violation 类型选择 forced tools、追加 system message，并在超过重试次数后返回可信错误。

[待添加：Verifier / Recovery 关键代码截图]

4.4 AI IDE 使用

项目开发过程中使用 AI 编码工具辅助进行代码阅读、架构设计、测试分析和文档整理。所有生成修改均通过本地代码检查、trace 和 benchmark 验证。

[待添加：AI IDE 使用截图，并说明一次具体开发闭环]

五、测试与评估

完整评测说明见 docs/evaluation.md。

5.1 Agent Benchmark

当前留存基线：

- 任务数：20。
- 通过：16。
- 成功率：80%。
- 平均步骤：2.4。
- 平均工具调用：1.35。

5.2 Memory Retrieval

- Cases : 20。
- Hits : 19。
- Recall@4 : 95%。
- MRR : 0.7667。

5.3 Replay 与失败分析

Trace replay 能重建 LLM 与工具调用顺序，并发现缺少文件证据的历史回复。Benchmark 将失败定位为回复断言、工具断言或产物内容问题，使优化具有可复现依据。

[待添加：latest.md、memory_retrieval_latest.md、trace/replay 截图]

六、系统升级与扩展

6.1 MCP-Compatible Tool Adapter

在不改变 Agent Loop 的前提下，为 ToolRegistry 增加 MCP-compatible adapter，使本地 Tool 与外部 MCP Tool 使用统一 schema 和执行入口。Verifier、Recovery 和 Trace 仍保留在 Runtime 控制面。

6.2 Attachment Index

增加独立附件索引，解决 session consolidation 后历史附件可见性不稳定的问题。

6.3 User-Scoped Memory

将当前全局长期记忆升级为按用户隔离的 memory store，支持真实多用户 QQBot 场景。

6.4 Workflow Planner 与沙箱

增加结构化多 Skill Workflow Planner，并使用 Docker 沙箱隔离高风险脚本和外部工具。

6.5 云端部署

容器化 Runtime 和 Channel 服务，提供可访问 Demo，并增加健康检查、限流、密钥管理和监控。

七、课程总结

[待填写：建议围绕以下内容完成，避免空泛。]

- 从“编写单个功能”到“设计 Agent Runtime 控制面”的工程思维变化。
- 对 Function Calling、ReAct、Memory、Skill 和 Harness 边界的理解。
- 从真实失败中学习：模型并不天然遵守工具协议，必须使用可验证证据。
- 对 SDD 的认识：需求、架构、接口、测试和 trace 应形成闭环。

- 对课程内容、实践方式和后续改进的建议。

参考资料与引用

- OpenAI-compatible Function Calling API。
- ReAct: Synergizing Reasoning and Acting in Language Models。
- Model Context Protocol。
- 本项目使用的开源 Python 库见 requirements.txt。
- [待补充：所有参考代码、论文和开源项目的具体链接与说明]